ANNEXURE A

Constraint-Based Problem Solving — Questions and Answers

Note: Question 2 does not specify the locations of the start, goal, or obstacles. To demonstrate the algorithm consistently, the solution below explicitly defines a sample 5×5 grid. If your instructor supplied a different grid diagram, its exact optimal path and cost may differ.

QUESTION 1 — Hospital Shift Assignment

Question: A hospital must assign doctors D1, D2, and D3 to Morning, Afternoon, and Night shifts, with these constraints: (1) D1 cannot work Night; (2) D2 must work before D3; (3) one doctor per shift; (4) D3 cannot work Morning; and (5) every doctor receives exactly one shift. Apply Backtracking Search, show intermediate steps and constraint checks, and state the final schedule.

Answer

This is a constraint satisfaction problem (CSP). The variables are the shifts for D1, D2, and D3. Each variable has the domain {Morning, Afternoon, Night}. The constraints require all shifts to be different, D1 ≠ Night, D3 ≠ Morning, and position(D2) < position(D3), where Morning = 1, Afternoon = 2, and Night = 3.

Backtracking steps and checks

Step	Assignment tried	Constraint check	Decision
1	D1 = Morning	D1 is not Night; valid so far.	Keep
2	D2 = Afternoon	Different from D1; position(D2)=2.	Keep
3	D3 = Night	Different from D1 and D2; D3 is not Morning; 2 < 3, so D2 is before D3.	Keep
4	Check all constraints	Each doctor has one shift; each shift has one doctor; all restrictions hold.	Solution found

Final valid schedule

Shift	Assigned doctor
Morning	D1
Afternoon	D2
Night	D3

Therefore, the valid assignment is D1–Morning, D2–Afternoon, and D3–Night. No backtracking is required on this variable/value order because the first complete assignment satisfies every constraint.

QUESTION 2 — Robot Navigation in a 5×5 Grid

Question: A robot must move from S to G in a 5×5 grid. It may move Up, Down, Left, or Right; each move costs 1; obstacles cannot be crossed; and Manhattan distance is specified as a heuristic. Use BFS to show node expansion, queue updates, and the optimal path and cost.

Answer and assumptions

BFS (Breadth-First Search) explores nodes level by level using a FIFO queue. Since every move costs 1, BFS finds a path with the minimum number of moves. Manhattan distance $h(n)=|\text{row}(n)-\text{row}(G)|+|\text{column}(n)-\text{column}(G)|$ is a useful heuristic for estimating distance, but standard BFS does not use heuristic values to order its queue. A heuristic-prioritized algorithm would be Greedy Best-First Search or A*. The steps below correctly use BFS and report Manhattan distances for reference.

Sample grid used because the question does not provide obstacle coordinates

S	.	#	.	.
.	.	#	.	#
#	.	.	.	.
.	#	#	#	.
.	.	.	.	G

Legend: S = start (row 1, column 1); G = goal (row 5, column 5); # = obstacle; . = free cell. Neighbours are considered in the order Right, Down, Left, Up. Coordinates are (row, column), starting at 1.

BFS queue / node expansion trace

Expanded node	Queue after expansion (front → rear)	Newly discovered nodes
(1,1)	(1,2), (2,1)	(1,2), (2,1)
(1,2)	(2,1), (2,2)	(2,2)
(2,1)	(2,2)	none
(2,2)	(3,2)	(3,2)
(3,2)	(3,3)	(3,3)
(3,3)	(3,4)	(3,4)
(3,4)	(3,5), (2,4)	(3,5), (2,4)
(3,5)	(2,4), (4,5)	(4,5)
(2,4)	(4,5), (1,4)	(1,4)
(4,5)	(1,4), (5,5)	(5,5)
(1,4)	(5,5), (1,5)	(1,5)
(5,5) G	(1,5)	none

Optimal path found

`(1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (3,5) → (4,5) → (5,5)`

Number of moves / path cost = 8. The path is: S (1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (3,5) → (4,5) → G (5,5).

Manhattan distance from a cell (r,c) to G=(5,5) is $h=|r-5|+|c-5|$. For example, h(S)=8 and h((4,5))=1. Because the goal is 8 Manhattan steps from the start and the obstacle layout allows a path of 8 moves, the path is optimal. BFS itself guarantees optimality here because all moves have equal cost; the heuristic is informational rather than used to prioritize the FIFO queue.

QUESTION 3 — Autonomous Rescue Robot

Question: A rescue robot must reach a survivor in a grid building. It can move up, down, left, or right; some cells are blocked; it can sense only adjacent cells; every move costs 1 and entering a risky zone adds 2; it must minimize cost safely and update its knowledge online. (a) Analyze the constraints. (b) Propose a suitable search strategy. (c) Explain intelligent agents, rationality, and environment type. (d) Justify the approach.

Answer

(a) Effect of the constraints

- **Four-direction movement:** The robot may consider only its immediate vertical and horizontal neighbours; diagonal shortcuts are not allowed.
- **Obstacles:** Blocked rooms must be excluded from the reachable-state graph.
- **Limited sensing:** The robot does not know the entire map initially. It must observe neighbouring cells, record what it learns, and avoid assuming that unseen cells are free.
- **Movement cost:** Each normal move costs 1. A move into a risky zone costs 1 + 2 = 3, so a longer safe route may be cheaper than a short risky route.
- **Dynamic knowledge:** The robot updates its map after each observation and replans if it discovers an obstacle or a safer route.
- **Safety and goal:** The robot must reach G while avoiding blocked cells and minimizing total accumulated cost.

(b) Proposed solution: Online Search Agent with Uniform Cost Search (UCS)

UCS is appropriate because step costs are not uniform: normal movement costs 1, while entering a risky cell costs 3. UCS expands the frontier state with the smallest accumulated path cost g(n), rather than the state with the fewest steps.

- **1. Initialize:** Set the robot's current position to S. Store a partial map with known free, blocked, and risky cells. Set g(S)=0.
- **2. Sense:** Observe the four adjacent cells, where the sensor can provide information. Update the map without treating unknown cells as confirmed free.
- **3. Generate safe successors:** Add only known traversable adjacent cells. Do not add blocked cells. For a normal cell, step cost = 1; for a risky cell, step cost = 3.
- **4. Update the frontier:** Insert each successor into a priority queue ordered by tentative total cost g. If a cheaper route to a state is found, update its cost and parent.
- **5. Choose the least-cost move:** Expand or move toward the frontier state with the lowest known cumulative cost, subject to the robot's safety policy.
- **6. Reobserve and replan:** After moving, sense adjacent cells again. If new information changes the map, update the graph and run UCS again from the current position.
- **7. Stop:** When G is reached, reconstruct the route using parent links and report its total cost.

Cost calculation

For a route containing n normal moves and r moves that enter risky zones, total cost = n + 3r. Equivalently, if the route has m total moves and r risky entries, cost = m + 2r. For example, 6 normal moves and 1 risky entry cost 6 + 3 = 9. The actual minimum-cost route cannot be calculated without a specific grid showing S, G, obstacles, and risky cells.

(c) AI concepts

Concept	Explanation in this scenario
Intelligent agent	The robot perceives its surroundings through sensors, maintains an internal map, chooses actions, and moves to reach the survivor.
Rationality	A rational robot selects actions that maximize expected success while minimizing cumulative cost and respecting safety, based on the information available at that time.

Concept	Explanation in this scenario
Environment type	Partially observable (only adjacent cells sensed); sequential (each move affects later choices); dynamic/unknown (new obstacles or risks may be discovered); discrete (grid cells and actions); multi-cost (normal and risky cells have different costs). It is single-agent unless other independent agents are included.
Online search	The robot discovers the environment while acting, updates its map, and replans rather than relying on a fully known map from the start.

(d) Why UCS with online replanning is suitable

- **Cost-optimality:** UCS finds a least-cost path when step costs are non-negative, including the extra penalty for risky zones.
- **Safety:** Blocked cells are never treated as valid successors; the robot can use a policy that avoids unknown or high-risk cells until sensed.
- **Adaptability:** Online sensing and replanning allow the robot to respond to newly discovered obstacles and risk information.
- **Correct fit:** BFS optimizes number of moves only when all moves have equal cost. Here, risky entries cost more, so UCS is more suitable than ordinary BFS.

Conclusion: Use an online search agent that repeatedly senses nearby rooms and applies Uniform Cost Search to the currently known map. It balances safe navigation with minimum accumulated movement cost. A concrete route and numerical final cost require the actual building grid and risk-zone locations.